Python Metaprogramming

Solutions to the six exercises

Each solution file (for example `ex1_metadata_introspection.py`) contains the code below followed by the same test code as the starter file, and produces the output shown.

Exercise 1: Dynamic Metadata Introspection: Solution

Revised notes, slides 3 and 4

Objective: Use the attributes every class carries (__name__, __bases__, __mro__ and __dict__) to describe any class at runtime.

Code solution:

```
def inspect_class(cls):
    """Return a summary dictionary describing any class."""
    attributes = [
        attr for attr in cls.__dict__      # only names defined in this class
        if not attr.startswith("__")       # skip Python's own dunder names
    ]
    return {
        "name": cls.__name__,
        "bases": [base.__name__ for base in cls.__bases__],
        "mro": [klass.__name__ for klass in cls.__mro__],
        "attributes": attributes,
    }
```

Explanation: `cls.__name__`, `cls.__bases__` and `cls.__mro__` describe the class and its hierarchy; converting each class to its __name__ makes the result easy to read. `cls.__dict__` holds only the names defined in this class (not inherited ones such as `category`), and filtering out names that start with "__" removes the entries Python adds automatically.

Output:

```
Class name: Child
Bases: ['Parent']
MRO: ['Child', 'Parent', 'object']
Own attributes: ['run', 'value']
```

Exercise 2: Dynamic Configuration Wrapper: Solution

Revised notes, slides 5 and 6

Objective: Use __getattr__ to read a dictionary with dot notation, including nested sections.

Code solution:

```
class ReadOnlyConfig:
    def __init__(self, data):
        # Our own __setattr__ blocks writes, so store _data directly
        object.__setattr__(self, "_data", dict(data))

    def __getattr__(self, name):
        # Called only when normal lookup fails.
        # Private names are never config keys; refusing them here also
        # prevents endless recursion if _data does not exist yet
        # (for example while the object is being copied).
        if name.startswith("_"):
            raise AttributeError(name)
        if name not in self._data:
            raise AttributeError(f"No setting called '{name}'")
        value = self._data[name]
        if isinstance(value, dict):
            return ReadOnlyConfig(value)    # wrap nested sections too
        return value

    def __setattr__(self, name, value):
        raise AttributeError("ReadOnlyConfig is read-only.")
```

Explanation: `__getattr__` runs only when normal lookup fails, so it can fetch the missing name from `self._data`. A nested dictionary is wrapped in a new `ReadOnlyConfig`, which is what makes `config.database.host` work. Names starting with "_" are refused straight away: they are never configuration keys, and the check stops `self._data` from calling `__getattr__` again, forever, when `_data` does not exist yet (for example while the object is being copied with `copy.copy()`).

Output:

```
Debug: True
Database host: localhost
Database port: 5432
Missing key: No setting called 'timeout'
Write blocked: ReadOnlyConfig is read-only.
```

Exercise 3: Strict Attribute Validation: Solution

Revised notes, slides 7 and 8

Objective: Use __setattr__ to check and tidy every value before it is stored.

Code solution:

```
class StrictEmployee:
    def __init__(self, name, salary):
        self.name = name          # both assignments go through __setattr__
        self.salary = salary

    def __setattr__(self, name, value):
        if name == "name":
            if not isinstance(value, str):
                raise TypeError("Employee name must be a string")
            value = value.strip()                  # normalise
            if not value:
                raise ValueError("Employee name cannot be empty")
        elif name == "salary":
            if not isinstance(value, (int, float)):
                raise TypeError("Salary must be a number")
            if value < 0:
                raise ValueError("Salary cannot be negative")
            value = float(value)                   # normalise
        # self.name = value here would call __setattr__ again forever
        object.__setattr__(self, name, value)
```

Explanation: Every assignment, including those in `__init__`, passes through `__setattr__`, so this one method guards the whole object. The value is checked, then normalised (spaces stripped, salary converted to float), and finally stored with `object.__setattr__`. Writing `self.name = value` inside `__setattr__` would call the method again, forever.

Output:

```
Employee registered: Alice, salary: 75000.0
ValueError: Employee name cannot be empty
ValueError: Salary cannot be negative
TypeError: Employee name must be a string
```

Exercise 4: Generating a Class with type(): Solution

Revised notes, slides 9 and 10

Objective: Use the three-argument form of type() to build a new class while the program runs.

Code solution:

```
def class_factory(class_name, fields):
    """Build a new class whose __init__ sets the given fields in order."""

    def __init__(self, *args):
        if len(args) != len(fields):
            raise TypeError(
                f"{class_name} expects {len(fields)} values, got {len(args)}")
        for field, value in zip(fields, args):
            setattr(self, field, value)

    def __repr__(self):
        values = ", ".join(f"{f}={getattr(self, f)!r}" for f in fields)
        return f"{class_name}({values})"

    namespace = {"__init__": __init__, "__repr__": __repr__}
    return type(class_name, (), namespace)    # name, parents, namespace
```

Explanation: `__init__` and `__repr__` are ordinary functions defined inside the factory, so they can use `class_name` and `fields`. They are placed in a namespace dictionary and passed to `type(class_name, (), namespace)`, which builds the class exactly as a `class` statement would.

Output:

```
Class name: Point3D
Coordinates: 10 20 30
Object: Point3D(x=10, y=20, z=30)
Error: Point3D expects 3 values, got 2
```

Exercise 5: Dynamic API Method Generator: Solution

Revised notes, slides 11 and 12

Objective: Generate one method per endpoint from a list of names, instead of writing each method by hand.

Code solution:

```
class ApiClient:
    def __init__(self, base_url):
        self.base_url = base_url

    def mock_get(self, endpoint):
        return f"GET request succeeded for {self.base_url}/{endpoint}"


def make_method(endpoint):
    """Return a method that calls mock_get for one fixed endpoint.

    Each call to make_method creates a new closure, so every method
    remembers its own endpoint. Defining the method directly inside the
    loop would make every method use the LAST endpoint in the list.
    """
    def method(self):
        return self.mock_get(endpoint)
    method.__name__ = f"get_{endpoint}"
    return method


def generate_api_client(endpoints):
    """Return a subclass of ApiClient with one get_<endpoint> method each."""
    namespace = {f"get_{ep}": make_method(ep) for ep in endpoints}
    return type("DynamicApiClient", (ApiClient,), namespace)
```

Explanation: Each call to `make_method` creates a new function that remembers its own `endpoint` (a closure). If the methods were defined directly inside a loop, they would all look up the loop variable when called, and every method would use the last endpoint in the list (late binding). The methods are collected in a namespace dictionary and passed to `type()` together with `(ApiClient,)` as the parent.

Output:

```
Users API: GET request succeeded for https://api.system.internal/users
Orders API: GET request succeeded for https://api.system.internal/orders
Generated methods: ['get_orders', 'get_users']
```

Exercise 6: Metaclass Interface Verifier: Solution

Revised notes, slides 13 to 16

Objective: Use a metaclass to reject badly formed classes at the moment they are defined.

Code solution:

```
class InterfaceVerifier(type):
    def __new__(mcls, name, bases, namespace):
        # Runs when each class statement is executed, before the class exists.
        # A class with no parents (BaseComponent) is the base: skip the checks.
        if bases:
            doc = namespace.get("__doc__")
            if not doc or not doc.strip():
                raise TypeError(
                    f"Class '{name}' must define a non-empty docstring.")
            if not callable(namespace.get("validate")):
                raise TypeError(
                    f"Class '{name}' must implement a 'validate(self)' method.")
        return super().__new__(mcls, name, bases, namespace)


class BaseComponent(metaclass=InterfaceVerifier):
    pass


# Extension: the same rules with __init_subclass__, no metaclass needed
class SimpleBase:
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        if not (cls.__doc__ or "").strip():
            raise TypeError(f"Class '{cls.__name__}' must define a non-empty docstring.")
        if not callable(cls.__dict__.get("validate")):
            raise TypeError(f"Class '{cls.__name__}' must implement a 'validate(self)' method.")
```

Explanation: The metaclass's `__new__` runs each time a `class` statement is executed, while the class body is still a plain dictionary, so a faulty class is rejected before it exists. Testing `if bases:` skips only the class with no parents; checking names instead would let any subclass called "Base" escape the rules. The extension shows the simpler alternative from the notes: `__init_subclass__` applies the same checks with no metaclass.

Output:

```
DatabaseComponent created successfully.
Docstring check failed correctly: Class 'BrokenDoc' must define a non-empty docstring.
Method check failed correctly: Class 'BrokenMethod' must implement a 'validate(self)' method.
No loophole: Class 'Base' must define a non-empty docstring.
Extension check: Class 'AlsoBroken' must implement a 'validate(self)' method.
```